Consolidação de ADRs: Projeto Busca Peça

Architecture Decision Records do marketplace de peças automotivas Busca Peça. Versão revisada para ficar coerente com o DER, o diagrama de casos de uso v2, a especificação v2 e as documentações de banco, back-end e front-end.

Versão: 2.0 **Data:** 01/11/2026 **Substitui:** ADRs_Busca_Peca.pdf (v1) **ADRs:** 15

Cada ADR segue a estrutura definida pela disciplina: **Status, Contexto, Decisão, Alternativas consideradas e Consequências**. Nesta versão, cada registro também traz **Relacionados** (casos de uso e artefatos afetados), para facilitar a rastreabilidade.

Legenda de status: Aceito (decisão tomada e já refletida nos artefatos) · Proposto (aguarda validação) · Substituído (trocado por outro ADR). **Marcadores:** Revisado na v2 conteúdo alterado; Novo na v2 registro criado nesta versão.

Índice

ID	Decisão	Status	Versão 2
ADR-001	API REST em Java (Spring Boot) como único ponto de acesso ao banco	Aceito	Revisado
ADR-002	Consulta opcional à base externa de códigos para autopreenchimento	Aceito	Revisado
ADR-003	Monolito modular em vez de microsserviços	Aceito	Revisado
ADR-004	Banco de dados relacional MySQL 8 com migrações Flyway	Aceito	Revisado
ADR-005	Aplicação web responsiva, pensada primeiro para o celular	Aceito	Revisado
ADR-006	Autenticação com hash BCrypt e JWT em cookie HttpOnly; autorização por papéis	Aceito	Revisado
ADR-007	Conta de usuário única com papéis de Comprador e Vendedor	Aceito	Revisado
ADR-008	Pagamento online integrado fora do escopo do MVP	Aceito	Revisado
ADR-009	Um pedido por loja a partir de um carrinho multilojas	Aceito	Revisado
ADR-010	Imagens armazenadas fora do banco de dados	Aceito	Revisado
ADR-011	Busca com recursos nativos do MySQL (FULLTEXT e índices) no MVP	Aceito	Revisado
ADR-012	Integrações externas isoladas, com timeout e alternativa	Aceito	Revisado
ADR-013	Conformidade com a LGPD desde a concepção	Aceito	Revisado
ADR-014	Front-end em HTML, CSS e JavaScript sem framework e sem etapa de build	Aceito	Novo
ADR-015	Repositórios separados para front-end e back-end, com contrato OpenAPI	Aceito	Novo

O que mudou da versão 1 para a versão 2

- **ADR-004:** a v1 sugeria PostgreSQL; a v2 registra o **MySQL 8**, já adotado nas documentações do banco e do back-end, e cita o PostgreSQL como alternativa considerada.
- **ADR-011:** a busca textual passa a ser descrita com os recursos do MySQL (índices FULLTEXT e collation que ignora acentos) e não mais do PostgreSQL.
- **ADR-014 e ADR-015 criados:** as documentações do front-end e do back-end já citavam esses números, mas eles não existiam na consolidação.
- **Status:** todos os ADRs passam a **Aceito**, porque as decisões já estão implementadas nas documentações de banco, back-end e front-end e no protótipo. Se a equipe discordar de algum, basta voltar o status para Proposto.
- **Detalhes técnicos consolidados:** tecnologias (Spring Boot, Flyway, Resilience4j, Bootstrap, Alpine.js), timeouts por integração, regras do cookie, CSRF e expiração do token, reserva de estoque na finalização e anonimização na exclusão de conta.
- **Relacionados:** cada ADR agora aponta os casos de uso e os artefatos que dependem dele. A tabela de impactos no final mostra o que já foi atualizado e o que ainda falta.

Registros de decisão

ADR-001: API REST em Java como único ponto de acesso ao banco de dados

Aceito Revisado na v2

Contexto: O Busca Peça tem um aplicativo web (front-end) e um banco com dados sensíveis de usuários, lojas, peças e pedidos. Se o front-end acessasse o banco diretamente, qualquer mudança nas tabelas quebraria as telas, e regras como "só o dono da loja edita suas peças" ficariam espalhadas, sem um ponto central de garantia.

Decisão: Uma API REST em **Java 21 com Spring Boot 3** é a intermediária obrigatória entre o front-end e o banco. O front-end nunca acessa o banco. Toda leitura e escrita passa por requisições JSON em rotas `/api/...`.

- Regras de negócio e verificação de dono ficam na camada de serviço, nunca no controller, no banco ou no front-end.
- A API nunca devolve entidades do banco; sempre devolve DTOs, sem senha, hash ou dados internos.
- Respostas de erro seguem um formato único (data, código, mensagem em português, rota e campos inválidos) e os códigos 400, 401, 403, 404, 409 e 500.
- Listagens são sempre paginadas (padrão de 20 itens, máximo de 100).

Alternativas consideradas: acesso direto do front-end ao banco (descartada por segurança e acoplamento); Backend-as-a-Service (descartada por limitar o controle das regras de negócio).

Consequências

- **Positivas:** regras concentradas em um só lugar; o banco pode mudar de estrutura sem afetar o front-end, desde que o contrato da API seja mantido; novos clientes (como um app mobile) podem usar a mesma API.
- **Negativas:** uma camada de rede a mais (Web → API → Banco) aumenta um pouco a latência; a API vira ponto único de falha.

Relacionados: ADR-003, ADR-006, ADR-015 · C4 (Zoom 2) · Documentação do Back-end.

ADR-002: Consulta opcional à base externa de códigos para autopreenchimento

Aceito Revisado na v2

Contexto: A peça pode ser cadastrada preenchendo tudo à mão ou informando apenas o código de identificação (OEM, EAN ou SKU). Para quem cadastra muitas peças excedentes, o preenchimento manual é lento e sujeito a erros. Existe uma base de códigos mantida por terceiros que pode fornecer esses dados.

Decisão: O módulo `integracao/codigopeca` da API faz uma consulta **opcional** à base externa. Se o vendedor informa o código, o sistema tenta buscar os dados e pré-preencher o formulário para revisão. A consulta tem **timeout de 3 segundos** (ADR-012).

- **Dados encontrados:** o formulário é pré-preenchido e o vendedor revisa antes de publicar.
- **Dados incompletos:** os campos faltantes ficam destacados para o vendedor completar.
- **Código não encontrado, erro ou timeout:** o formulário é liberado para preenchimento manual, com aviso discreto e mantendo o código informado.

Alternativas consideradas: exigir sempre o preenchimento manual (descartada por ser lenta); depender obrigatoriamente da base externa (descartada pelo risco de indisponibilidade).

Consequências

- **Positivas:** cadastro mais rápido para quem conhece o código, menos erros e cadastro em massa facilitado. O sistema continua funcionando se a base falhar, porque o autopreenchimento é um atalho, não uma exigência.
- **Negativas:** dependência de um serviço fora do controle da equipe, sem garantia de disponibilidade ou de qualidade dos dados; é preciso tratar e testar os três cenários de falha.

Relacionados: BP1-9, BP1-63, BP1-45 · ADR-012 · Diagrama de atividades B · Fluxo B do protótipo.

ADR-003: Monolito modular em vez de microsserviços

Aceito Revisado na v2

Contexto: A equipe tem 4 pessoas, prazo acadêmico restrito e orçamento zero para infraestrutura. O C4 (Zoom 3) já mostra uma única API dividida em componentes.

Decisão: A API é um **monolito modular**: uma única aplicação Spring Boot, implantada como uma unidade, com pacotes organizados por domínio dentro de `br.com.buscapeca`:

`usuario` · `loja` · `peca` · `catalogo` · `busca` · `carrinho` · `pedido` · `comunicacao` · `posvenda` · `notificacao` · `administracao` · `integracao`, mais as pastas transversais `config`, `seguranca` e `compartilhado`.

- Cada módulo é dono das suas tabelas: só ele grava nelas. Outro módulo que precise desses dados chama o serviço do dono, nunca o repositório.
- Todo módulo tem as mesmas camadas: `controller`, `service`, `repository`, `entity`, `dto`, `mapper` e `enums`.
- Os módulos de fase 2 (`pagamento` e `comissao`) não são criados no MVP (ADR-008).

Alternativas consideradas: microsserviços (descartada: complexidade operacional e de comunicação incompatível com a equipe e o prazo); monolito sem divisão por domínio (descartada: dificulta o trabalho em paralelo e uma futura separação).

Consequências

- **Positivas:** desenvolvimento, teste e implantação simples, com um repositório e um processo de deploy; as fronteiras por domínio permitem extrair serviços no futuro.
- **Negativas:** o escalonamento é do sistema inteiro; exige disciplina para não misturar as fronteiras entre módulos.

Relacionados: ADR-001, ADR-012 · C4 (Zoom 3) · Documentação do Back-end (módulos por domínio).

ADR-004: Banco de dados relacional MySQL 8 com migrações Flyway

Aceito Revisado na v2

Contexto: O modelo de dados (DER) é fortemente relacional: usuários e papéis, lojas, peças, compatibilidade com veículos, categorias hierárquicas, compras, pedidos e itens. Há integridade referencial importante, transações em estoque e pedidos e consultas com filtros combinados. A v1 deste ADR sugeria PostgreSQL e deixava a escolha para a equipe; a documentação do banco e a do back-end foram escritas com MySQL.

Decisão: Usar **MySQL 8** com motor **InnoDB** e charset `utf8mb4` com collation `utf8mb4_0900_ai_ci` (buscas ignoram acento e maiúsculas), no schema `buscapeca`.

- O esquema segue o DER (46 tabelas: 43 no MVP e 3 de fase 2), com chaves estrangeiras, restrições de unicidade e verificações (CHECK) de valores.
- As tabelas são criadas e alteradas **somente por migrações Flyway** versionadas; uma migração que já rodou nunca é editada.
- Acesso pela API com Spring Data JPA (`ddl-auto=validate`). Localmente, o MySQL roda em Docker Compose.
- Usuário, loja e peça usam exclusão lógica, porque pedidos antigos continuam apontando para eles.

Alternativas consideradas: PostgreSQL (sugerido na v1; também atenderia, mas a equipe padronizou o MySQL 8 nas documentações e no ambiente local); banco NoSQL de documentos (descartada: muitos relacionamentos e necessidade de transações em estoque e pedidos).

Consequências

- **Positivas:** integridade dos dados, transações com bloqueio de linha para estoque e pedidos, consultas com filtros complexos e histórico de mudanças do esquema pelo Flyway.
- **Negativas:** mudanças de esquema exigem migrações e combinação prévia dos números entre os integrantes; a escalabilidade horizontal é mais difícil, mas irrelevante para o MVP; a busca textual nativa do MySQL é mais limitada que a de um motor dedicado (ver ADR-011).

Relacionados: ADR-009, ADR-011, ADR-013 · DER · Documentação do Banco de Dados (MySQL) · Documentação do Back-end.

ADR-005: Aplicação web responsiva, pensada primeiro para o celular

Aceito Revisado na v2

Contexto: O escopo do MVP (Duplo Diamante e SWOT) é uma plataforma web. O app mobile aparece só como crescimento futuro. Mecânicos e balconistas usam muito o celular na oficina e no balcão.

Decisão: Entregar o MVP como **aplicação web responsiva**, desenhada primeiro para o celular e ampliada para tablet e desktop (pontos de quebra do Bootstrap: abaixo de 768 px, de 768 a 991 px e a partir de 992 px). O app nativo fica no backlog e reutilizará a mesma API (ADR-001).

Alternativas consideradas: app nativo desde o início (descartada: custo e prazo); PWA (fica como evolução futura).

Consequências

- **Positivas:** uma única base de código de front-end, entrega mais rápida e sem publicação em lojas de aplicativos.
- **Negativas:** acesso limitado a recursos nativos (câmera e localização dependem do navegador) e experiência móvel inferior à de um app nativo.

Relacionados: ADR-014 · Protótipo Figma (desktop e mobile) · Documentação do Front-end.

ADR-006: Autenticação com hash BCrypt e JWT em cookie HttpOnly; autorização por papéis

Aceito Revisado na v2

Contexto: O sistema guarda dados pessoais e empresariais (CPF/CNPJ, endereço) e tem áreas restritas (Minha Loja e área administrativa). Login, recuperação de senha e permissões administrativas (BP1-40, BP1-41, BP1-56) exigem identificação segura e controle de acesso.

Decisão:

- **Senha:** guardada apenas como hash **BCrypt**, nunca em texto puro.
- **Sessão:** após o login, a API emite um **JWT** com o id do usuário e seus papéis, gravado em **cookie HttpOnly** (com `Secure` em produção). O front-end nunca lê nem guarda o token. Validade padrão de 120 minutos (`JWT_EXPIRACAO_MINUTOS`).
- **CSRF:** como o token vai em cookie, requisições POST, PUT, PATCH e DELETE exigem o token CSRF obtido em `GET /api/auth/csrf`. O CORS libera só a origem do front-end, com credenciais.
- **Autorização:** verificada na API a cada requisição, por papel (Comprador, Vendedor, Administrador) e por propriedade ("só o dono da loja edita suas peças"). A proteção de páginas no front-end serve só à experiência do usuário.
- **Papéis internos:** o papel Administrador nunca é escolhido no cadastro público; é atribuído por outro administrador. A área administrativa usa papéis internos com permissões próprias (por exemplo, Administrador geral, Moderador e Suporte), por meio de PAPEL, PERMISSAO e PAPEL_PERMISSAO.
- **Recuperação de senha:** link com token de uso único, validade de 30 minutos e apenas o hash guardado (TOKEN_VERIFICACAO), enviado por e-mail.

Alternativas consideradas: sessão no servidor (viável, mas menos adequada a um futuro app mobile); JWT guardado no `localStorage` (descartada: exposto a ataques XSS); provedor externo de identidade (descartada por custo e dependência).

Consequências

- **Positivas:** segurança adequada ao tipo de dado, regra de acesso centralizada na API e token protegido contra leitura por scripts.
- **Negativas:** exige tratar CSRF, expiração e renovação do token; front-end e API precisam estar sob o mesmo domínio para o cookie funcionar (ADR-015); revogar um token antes do vencimento exige checar o status da conta (suspensa ou inativa) a cada requisição.

Relacionados: BP1-2, 40, 41, 49, 56, 62 · ADR-007, ADR-013, ADR-015 · Diagramas de atividades D1 e D2 · Fluxos D e E do protótipo.

ADR-007: Conta de usuário única com papéis de Comprador e Vendedor

Aceito Revisado na v2

Contexto: O SWOT apresenta como diferencial que a mesma oficina pode comprar e vender. A versão inicial do BP1-2 obrigava a escolher um único tipo no cadastro, e o campo `tipo_perfil` do DER guardava um único valor, o que contradizia a proposta de valor.

Decisão: Cada pessoa ou empresa tem **uma única conta**. Toda conta nasce com o papel **Comprador**. O papel **Vendedor** é ativado ao criar a loja ("Minha Loja", BP1-67). O papel Administrador é exclusivo e interno (ADR-006).

- No modelo, `tipo_perfil` foi removido de USUARIO; os papéis ficam em PAPEL e USUARIO_PAPEL, e a existência de LOJA caracteriza o vendedor.
- O cadastro não pede tipo de perfil; CPF/CNPJ só é pedido ao criar a loja (minimização, ADR-013).
- Usuário sem loja que abre uma página da Minha Loja é levado a "Criar loja".

Alternativas consideradas: contas separadas por tipo (descartada: obrigaria a oficina a ter dois cadastros).

Consequências

- **Positivas:** coerência com a proposta de valor, cadastro único e maior engajamento.
- **Negativas:** telas e menus precisam variar conforme os papéis; a especificação, o DER, o diagrama de classes e o de casos de uso tiveram de ser ajustados.

Relacionados: BP1-2, 41, 60, 67 · DER (PAPEL, USUARIO_PAPEL) · Diagrama de casos de uso v2 · Diagramas de atividades B e D1.

ADR-008: Pagamento online integrado fora do escopo do MVP

Aceito Revisado na v2

Contexto: O SWOT lista "sem checkout/pagamento integrado" como limitação do MVP e prevê o gateway de pagamento como evolução futura. As especificações BP1-24, BP1-25, BP1-34 e BP1-64 e o DER (PAGAMENTO, COMISSAO, CUPOM) modelavam checkout com Pix, cartão, cupons e comissão, o que excede a capacidade da equipe e o prazo.

Decisão: No MVP, a plataforma conecta comprador e vendedor e registra o pedido (reserva da peça, status, código de retirada e chat). **Pagamento, frete e retirada são combinados diretamente entre as partes, fora da plataforma.** A loja apenas informa as formas que aceita (LOJA_METODO_PAGAMENTO), usadas no filtro BP1-14.

- Ficam na **fase 2**: gateway de pagamento (BP1-25), cupom de desconto (BP1-34) e comissões (BP1-64), com as tabelas PAGAMENTO, CUPOM e COMISSAO.
- Nenhum valor é cobrado nem estornado pela plataforma no MVP; reembolsos, quando houver, são combinados entre as partes.

Alternativas consideradas: integrar o gateway desde o início (descartada: custo, risco regulatório e tempo).

Consequências

- **Positivas:** escopo viável para 4 pessoas e menor risco financeiro e legal no MVP.
- **Negativas:** possível desconfiança do comprador e dificuldade em fechar vendas (já apontado no SWOT); sem pagamento na plataforma, não há comissão automática nem receita direta.

Relacionados: BP1-14, 24, 25, 26, 30, 34, 36, 64 · DER (fase 2) · Diagramas de atividades A e C.

ADR-009: Um pedido por loja a partir de um carrinho multilojas

Aceito Revisado na v2

Contexto: O carrinho pode ter peças de lojas diferentes, principalmente com o Agrupador de Pedidos Multilojas (BP1-5). Cada loja aceita ou recusa o seu pedido de forma independente (BP1-26) e tem o seu próprio frete e retirada.

Decisão: Ao finalizar a compra, o sistema cria uma **COMPRA** e a divide em **um PEDIDO por loja**. Cada pedido tem o seu status, frete, código de retirada, chat, avaliação, devolução e fluxo de aceite, recusa e cancelamento. O comprador é obtido pela COMPRA.

- **Estoque:** a finalização é uma única transação. As linhas das peças são bloqueadas, o estoque é baixado, a compra, os pedidos e os itens são gravados e o carrinho é esvaziado. Se faltar estoque de qualquer peça, nada é gravado.
- **Recusa e cancelamento** devolvem as quantidades ao estoque.
- **Status do pedido:** Aguardando aceite → Aceito → Em preparação → Pronto para retirada ou Enviado → Concluído; ou Recusado / Cancelado.

Alternativas consideradas: pedido único com várias lojas (descartada: aceite parcial e frete por loja tornam o modelo confuso); baixar o estoque só no aceite da loja (descartada: duas pessoas poderiam pedir a última unidade ao mesmo tempo).

Consequências

- **Positivas:** cada vendedor gerencia apenas os seus pedidos e a última unidade nunca é vendida duas vezes.
- **Negativas:** o comprador vê vários pedidos em uma compra, por isso o histórico agrupa por número de compra; uma peça fica reservada enquanto a loja não responde.

Relacionados: BP1-5, 20, 24, 26, 27, 30 · DER (COMPRA, PEDIDO) · Documentação do Banco (regras de transação) · Diagramas de atividades A e C.

ADR-010: Imagens armazenadas fora do banco de dados

Aceito Revisado na v2

Contexto: Cada anúncio tem várias fotos (BP1-47) e as lojas têm logo e banner (BP1-55). Guardar arquivos binários no banco aumentaria o tamanho, o custo e o tempo de backup; o DER já usa

campos de URL.

Decisão: Os arquivos ficam fora do banco, e o banco guarda apenas a URL. No MVP, as imagens são gravadas numa pasta do servidor (`UPLOAD_DIRETORIO`), por meio do adaptador `integracao/armazenamento`, o que permite trocar depois por um serviço de objetos sem mudar os módulos de negócio.

- A API valida extensão, tipo do arquivo e tamanho máximo antes de aceitar o upload: **JPG, PNG ou WebP, até 5 MB por arquivo**. O front-end confere o mesmo antes do envio.
- A pasta `uploads/` do ambiente local fica fora do Git.

Alternativas consideradas: salvar as imagens como BLOB no banco (descartada por desempenho e custo); serviço de armazenamento em nuvem desde o início (adiada: custo e configuração).

Consequências

- **Positivas:** banco leve, consultas rápidas e possibilidade de usar CDN no futuro.
- **Negativas:** mais um componente a manter; é preciso cuidar do backup das imagens e da limpeza de arquivos órfãos.

Relacionados: BP1-47, 55, 61 · ADR-012 · Diagrama de atividades B.

ADR-011: Busca com recursos nativos do MySQL no MVP

Aceito · Revisado na v2

Contexto: A busca é a funcionalidade central: por nome, código, categoria, veículo e vários filtros (condição, origem, localização, disponibilidade, avaliação, forma de pagamento). Motores dedicados (como Elasticsearch) aumentam a infraestrutura e o custo.

Decisão: No MVP, a busca usa os recursos do próprio MySQL (ADR-004):

- Índices **FULLTEXT** em título e descrição da peça e índices comuns nos códigos OEM, EAN e SKU (migração V5).
- A collation `utf8mb4_0900_ai_ci` faz a busca ignorar acentos e maiúsculas.
- A distância é calculada a partir da latitude e da longitude das lojas e do comprador (colunas `DECIMAL(10,7)`); endereço sem coordenadas fica fora do filtro por distância.
- Busca por diagrama técnico (BP1-12) e busca por foto (BP1-16) ficam na **fase 2**; as tabelas DIAGRAMA_TECNICO e HOTSPOT entram numa migração posterior.

Alternativas consideradas: motor de busca dedicado (descartada: custo e complexidade neste estágio).

Consequências

- **Positivas:** nenhum serviço adicional, simples de manter e suficiente para o volume do MVP.
- **Negativas:** relevância e tolerância a erros de digitação limitadas; se o catálogo crescer muito, será necessário migrar para um motor dedicado.

Relacionados: BP1-4, 6, 7, 8, 10, 12, 13, 15, 16, 17, 18 · ADR-004 · Diagrama de atividades A.

ADR-012: Integrações externas isoladas, com timeout e alternativa

Aceito · Revisado na v2

Contexto: Além da base de códigos (ADR-002), o sistema depende de serviços de terceiros: cálculo de frete, geolocalização, envio de e-mail e armazenamento de imagens; no futuro, pagamento e reconhecimento de imagem. Nenhum deles está sob controle da equipe.

Decisão: Cada integração fica numa subpasta de `integracao/` com três partes: a interface (o que o sistema precisa), o adaptador (a chamada ao fornecedor) e os DTOs da resposta. Os módulos de negócio conhecem só a interface. Timeout, nova tentativa e alternativa são implementados com **Resilience4j**.

Integração	Usada por	Timeout	Se falhar
codigopeca	Autopreenchimento (BP1-9, 63)	3 s	Segue para o cadastro manual
frete	Checkout (BP1-23)	5 s	Exibe "frete a combinar"
geolocalizacao	CEP para coordenadas (BP1-7, 18)	3 s	Pede o CEP manual; sem coordenadas, o endereço fica fora do filtro de distância
email	Recuperação de senha e notificações (BP1-40, 51)	10 s	Registra a falha no log e permite reenviar
armazenamento	Fotos, logo e banner (BP1-47, 55)	—	Recusa o upload com mensagem clara

No ambiente de desenvolvimento, integrações sem conta configurada usam uma implementação falsa, ativada pelo perfil `dev`.

Alternativas consideradas: chamar os serviços externos direto nas regras de negócio (descartada: acoplamento e falhas em cascata).

Consequências

- **Positivas:** o núcleo do sistema continua operando com falha externa; trocar de fornecedor afeta só o adaptador; a equipe trabalha sem depender dos serviços reais.
- **Negativas:** mais código para tratar falhas e testar os cenários degradados.

Relacionados: BP1-7, 9, 18, 23, 33, 40, 47, 55, 63 · ADR-002, ADR-010 · C4 (Zoom 1) · Diagramas de atividades A, B e D2.

ADR-013: Conformidade com a LGPD desde a concepção

Aceito · Revisado na v2

Contexto: A plataforma trata dados pessoais e empresariais (nome, e-mail, telefone, CPF/CNPJ, endereços, localização, histórico de pedidos, mensagens e buscas salvas). A LGPD exige base legal, transparência e segurança. Os casos de uso já preveem termos e políticas (BP1-54) e logs de auditoria.

Decisão:

- **Minimização:** coletar só o necessário em cada tela; CPF/CNPJ apenas ao criar a loja.
- **Aceite versionado:** aceite dos Termos de uso e da Política de Privacidade no cadastro (caixa não marcada por padrão) e dos Termos do vendedor ao criar a loja, com versão e data em ACEITE_TERMO. Uma nova versão publicada pode exigir novo aceite no próximo acesso.
- **Localização:** pedir permissão explícita, explicando o motivo, com o CEP manual como alternativa.

- **Auditoria:** ações sensíveis (login, alteração de preço, exclusão de peça, ações administrativas) vão para LOG_AUDITORIA com parâmetros anonimizados.
- **Direitos do titular:** consultar, corrigir, exportar e pedir a inativação ou exclusão da conta pelo perfil.
- **Exclusão:** a conta não é apagada, porque há pedidos ligados a ela. O status vira INATIVO, os dados pessoais são substituídos por valores anônimos e endereços, favoritos, buscas salvas e tokens são apagados.
- **Segurança:** HTTPS obrigatório, hash de senha (ADR-006), nenhum dado pessoal no armazenamento do navegador e aviso de cookies no primeiro acesso.
- Não comercializar dados pessoais com terceiros.

Alternativas consideradas: tratar a conformidade só depois do MVP (descartada: retrabalho no modelo de dados e risco legal).

Consequências

- **Positivas:** menor risco legal, mais confiança dos usuários e atendimento ao entregável de LGPD da disciplina.
- **Negativas:** exige tabelas e telas adicionais (aceite, exclusão, exportação) e disciplina para não coletar além do necessário.

Relacionados: BP1-2, 7, 18, 49, 54, 60, 67 · DER (TERMO_USO, ACEITE_TERMO, LOG_AUDITORIA) · Documentação do Banco (exclusão de conta) · Fluxo D do protótipo.

ADR-014: Front-end em HTML, CSS e JavaScript sem framework e sem etapa de build

Aceito | Novo na v2

Contexto: A documentação do front-end já seguia esta decisão e citava o ADR-014, que não existia na consolidação. A equipe tem 4 pessoas, prazo curto e precisa entregar telas responsivas (ADR-005) que conversem com a API REST (ADR-001). Frameworks de SPA exigem aprendizado, ferramentas de build e mais configuração.

Decisão: O front-end é um site de várias páginas em **HTML5, CSS e JavaScript** (ES Modules nativos), **sem framework de SPA e sem etapa de build**.

- **Bootstrap 5.3** e Bootstrap Icons para layout e componentes, com um CSS próprio para a identidade visual (variáveis em `variaveis.css`).
- **Alpine.js 3** apenas nas telas que mudam durante a interação (filtros, carrinho, checkout, formulário de peça, conversas).
- Bibliotecas carregadas por CDN com versão fixa; nada a instalar.
- Somente o arquivo `api.js` conversa com a API; nenhuma regra de negócio fica no front-end.

Alternativas consideradas: SPA com React, Vue ou Angular (descartada: curva de aprendizado e etapa de build incompatíveis com o prazo); páginas geradas no servidor com Thymeleaf (descartada: acopla o front ao back-end e contraria o ADR-015).

Consequências

- **Positivas:** qualquer integrante consegue editar e rodar o site com um servidor local simples; implantação como arquivos estáticos; menos dependências.
- **Negativas:** componentes repetidos (cabeçalho, card, paginação) precisam ser montados por JavaScript próprio; não há tipagem nem roteamento prontos; depende da disponibilidade da

CDN.

Relacionados: ADR-001, ADR-005, ADR-006, ADR-015 · Documentação do Front-end.

ADR-015: Repositórios separados para front-end e back-end, com contrato OpenAPI

Aceito | Novo na v2

Contexto: As documentações do front-end e do back-end já citavam o ADR-015, que não existia na consolidação. Front-end e back-end usam tecnologias e ferramentas diferentes (IntelliJ e Maven de um lado; arquivos estáticos e Live Server do outro) e serão desenvolvidos em paralelo por integrantes diferentes.

Decisão:

- Dois repositórios: `buscapeca-backend` e `buscapeca-frontend`, cada um com branch `main` protegida, uma branch por funcionalidade e pull request revisado por outro integrante.
- O contrato entre os dois é a **documentação OpenAPI (Swagger)** gerada pela API com springdoc-openapi, em `/swagger-ui.html`. Os nomes finais das rotas valem conforme o Swagger.
- Em produção, front-end e API ficam em subdomínios do mesmo domínio (`app.` e `api.`), com HTTPS, para o cookie de login funcionar (ADR-006). Localmente, o front-end roda em `localhost:5500` e a API em `localhost:8080`.
- Segredos (senha do banco, chave do JWT, chaves de integração) ficam só em variáveis de ambiente do back-end, nunca no GitHub nem no front-end.

Alternativas consideradas: repositório único (monorepo) com as duas partes (descartada: mistura ferramentas e ciclos de entrega diferentes); front-end servido pela própria aplicação Spring (descartada: acopla a implantação dos dois).

Consequências

- **Positivas:** trabalho em paralelo com menos conflitos, implantação independente e contrato explícito entre as partes.
- **Negativas:** mudanças que afetam os dois lados exigem dois pull requests coordenados; é preciso configurar CORS e manter o Swagger atualizado.

Relacionados: ADR-001, ADR-006, ADR-014 · Documentações do Front-end e do Back-end.

Impactos nos demais artefatos

Situação conferida em 01/11/2026 contra os arquivos do projeto. **Atualizado** = o artefato já reflete o ADR. **Pendente** = ainda precisa de ajuste.

ADR	Artefato e ajuste	Situação
ADR-002, 012	BP1-9 e BP1-63: integração opcional, fluxos de timeout, serviço fora do ar e dados incompletos	**Atualizado**
ADR-003	C4 (Zoom 3): mostrar os módulos por domínio, incluindo Pedidos, Notificações, Administração e Integrações	**Pendente (conferir)**
ADR-004, 011	Documentações do banco e do back-end com MySQL 8, Flyway e índices FULLTEXT	**Atualizado**
ADR-006, 007	BP1-2 sem escolha de tipo; DER e diagrama de classes com papéis; diagrama de casos de uso v2	**Atualizado**
ADR-007	Especificação v2: trocar o ator "Fornecedor" por "Vendedor" em BP1-3, 26, 32, 40, 42, 44, 45, 50, 52, 55, 60, 63 e 65; no BP1-3, a pré-condição deve ser "usuário com loja criada"	**Pendente**
ADR-007	DER: incluir o BP1-67 (Criar Loja) na tabela de rastreabilidade e remover a nota de ajustes já feitos na especificação	**Pendente**
ADR-008	BP1-25, 34 e 64 marcados como fase 2; PAGAMENTO, CUPOM e COMISSAO como fase 2 no DER	**Atualizado**
ADR-009	DER com COMPRA; BP1-24 com divisão por loja; regras de transação na documentação do banco; diagramas de atividades A e C	**Atualizado**
ADR-009	Protótipo (Fluxo C): o texto "fica com 1 após o aceite" e "O estoque não foi alterado" na recusa contrariam a reserva na finalização. Ajustar para "estoque reservado na compra" e "estoque devolvido"	**Pendente**
ADR-010	Protótipo: o upload aceita só JPG e PNG, mas o ADR e o front-end aceitam também WebP. Alinhar o texto do protótipo	**Pendente**
ADR-011	DER: marcar DIAGRAMA_TECNICO e HOTSPOT como fase 2 (BP1-12 e BP1-16 já estão como fase 2 na especificação)	**Pendente**
ADR-012	C4 (Zoom 1): mostrar os serviços externos de frete, geolocalização, e-mail e base de códigos	**Pendente (conferir)**
ADR-013	DER com aceite de termos e log de auditoria; documentação do banco com a regra de exclusão da conta	**Atualizado**
ADR-014, 015	Documentações do front-end e do back-end (já citavam estes números)	**Atualizado**
Todos	Diagramas de atividades (A, B, C, D1, D2, E) criados seguindo os ADRs	**Atualizado**

"Conferir": os diagramas C4 do projeto são imagens e não puderam ser lidos automaticamente; vale abrir o arquivo e checar.